MATH70097 - Supervised Learning: Assessment 1

A technology company is running an online advertising campaign. Users are shown a banner while browsing the web. Some users click on the banner and then subscribe to the service. This event is called a conversion. Your task is to predict whether a conversion occurs for each user, based on information about the user, the current advertising campaign, and a previous related advertising campaign.

We approach this assignment in 4 parts:

1. **Exploratory Data Analysis and Preprocessing** — understand the structure and signals, detect issues (missing values, unusual values, imbalance, nonlinearity).
2. **Building Prediction Models** — train and compare Logistic Regression (with Ridge and Lasso variants), kNN, LDA, QDA, and Naive Bayes.
3. **Validation and Model Selection** — 10-fold stratified CV for hyperparameter tuning, Bootstrap for uncertainty quantification, and principled model selection via log-loss.
4. **Generating Test Predictions** — refit on the full training set and produce predicted probabilities.

Setup & Loading data

We import the relevant packages, set a seed, and load the data. The kernel used is the ISLP (Python 3.12.2) Kernel - which we setup in week 1 of the course.

In [24]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import StratifiedKFold, cross_val_score, cro
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, Qua
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import log_loss, confusion_matrix
from sklearn.calibration import calibration_curve
from sklearn.decomposition import PCA
from sklearn.base import clone
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid', palette='muted')
pd.set_option('display.max_columns', 25)
SEED = 42
np.random.seed(SEED)
```

In [25]:
```
train = pd.read_csv('train.csv')
test  = pd.read_csv('test.csv')
print(f"Training set : {train.shape[0]} rows, {train.shape[1]} columns")
print(f"Test set     : {test.shape[0]} rows, {test.shape[1]} columns")
train.head()
```

```
Training set : 31647 rows, 17 columns
Test set     : 13564 rows, 17 columns
```

Out[25]:

	age	job	marital	education	X2	X4	X3	X1	device	day	month
0	36	technician	single	tertiary	0	0	0	0	unknown	17	jur
1	56	entrepreneur	married	secondary	0	196	0	0	cellular	19	nov
2	46	blue-collar	married	secondary	0	0	1	0	unknown	5	jur
3	41	management	divorced	tertiary	0	3426	0	0	cellular	1	oct
4	38	blue-collar	married	secondary	0	0	1	0	unknown	20	may

Part 1: Explore the training data (EDA)

Following the data descriptions from the brief, as well as looking through our dataframe, we choose to split the variables into two types. Quantitative variables and Qualitative (categorical) variables.

In [26]:
```
# Define quantitative variables
quantitative = ['age', 'X4', 'day', 'time_spent', 'banner_views', 'days_e

# Define qualitative variables
qualitative = ['job', 'marital', 'education', 'X1', 'X2', 'X3', 'device',

# Print qualitative and quantitative variables

print("Quantitative variables:")
print(train[quantitative].dtypes)
print("\nQualitative variables:")
for col in qualitative:
    levels = train[col].unique()
    print(f"{col} : {len(levels)} levels - {list(levels)}")
```

```
Quantitative variables:
age                      int64
X4                       int64
day                      int64
time_spent               int64
banner_views             int64
days_elapsed_old       float64
banner_views_old         int64
dtype: object

Qualitative variables:
job : 12 levels - ['technician', 'entrepreneur', 'blue-collar', 'managemen
t', 'admin.', 'retired', 'housemaid', 'unknown', 'services', 'student', 'u
nemployed', 'self-employed']
marital : 3 levels - ['single', 'married', 'divorced']
education : 4 levels - ['tertiary', 'secondary', 'primary', 'unknown']
X1 : 2 levels - [np.int64(0), np.int64(1)]
X2 : 2 levels - [np.int64(0), np.int64(1)]
X3 : 2 levels - [np.int64(0), np.int64(1)]
device : 3 levels - ['unknown', 'cellular', 'telephone']
outcome_old : 4 levels - ['unknown', 'success', 'other', 'failure']
month : 12 levels - ['jun', 'nov', 'oct', 'may', 'jul', 'aug', 'feb', 'dec
', 'sep', 'jan', 'mar', 'apr']
```

We find that there are no null or NaN values in the dataset. However, for 'job', 'education', 'device' and 'outcome_old', we have "unknown" values to serve as a missing indicator. Similarly, for 'days_elapsed_old', the brief indicates values of -1 would mean the user never saw the old banner.

In [27]:

```
print("unknown values in training set:")
for x in ['job', 'education', 'device', 'outcome_old']:
    unknown_count = (train[x] == 'unknown').sum()
    percentage = (unknown_count / len(train)) * 100
    print(f"{x} : {unknown_count} ({percentage:.2f}%)")

print(f"\ndays_elapsed_old == -1 in train: {(train['days_elapsed_old'] ==
print(f"days_elapsed_old == -1 in test : {(test['days_elapsed_old'] == -1
```

```
unknown values in training set:
job : 203 (0.64%)
education : 1322 (4.18%)
device : 9065 (28.64%)
outcome_old : 25867 (81.74%)

days_elapsed_old == -1 in train: 0
days_elapsed_old == -1 in test : 0
```

Missing values takeaway

"unknown": we retain "unknown" as a factor level. The lack of a value itself provides us with information on users which were not exposed to the old banner.

'device': over 80% of the values are "unknown". We keep the variable but approach it carefully in model development as it carries limited signal to our predictors.

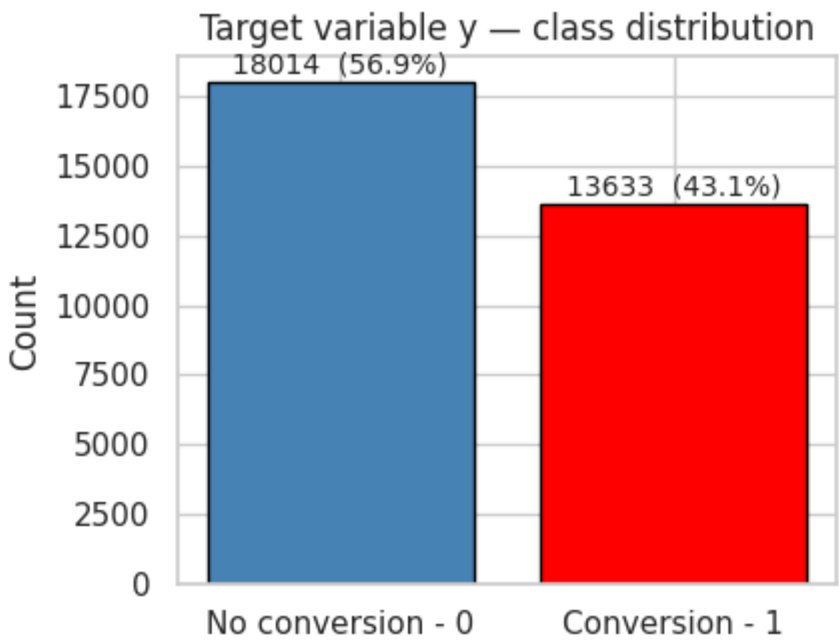
'days_elapsed_old': No -1 values are found in our dataset, so we can avoid handling this case specifically.

Numeric Variables: There are no missing numerical values, so no imputation is needed.

After checking for missing and unusual values, we check for imbalances.

In [28]:
```
rate = train['y'].mean()
print(f"Overall conversion rate (y=1) in training set: {rate:.4f}")
ratio = (train['y'] == 0).sum() / (train['y'] == 1).sum()
print(f"Class imbalance ratio (y=0 : y=1) in training set: {ratio:.2f} :

fig, ax = plt.subplots(figsize=(4.5, 3.5))
counts = train['y'].value_counts().sort_index()
bars = ax.bar(['No conversion - 0', 'Conversion - 1'], counts.values,
              color=['steelblue', 'red'], edgecolor='black')
for i, v in enumerate(counts.values):
    ax.text(i, v + 300, f"{v}  ({100*v/len(train):.1f}%)", ha='center', f
ax.set_ylabel('Count')
ax.set_title('Target variable y — class distribution')
plt.tight_layout()
plt.show()
```

```
Overall conversion rate (y=1) in training set: 0.4308
Class imbalance ratio (y=0 : y=1) in training set: 1.32 : 1
```



Imbalances takeaway

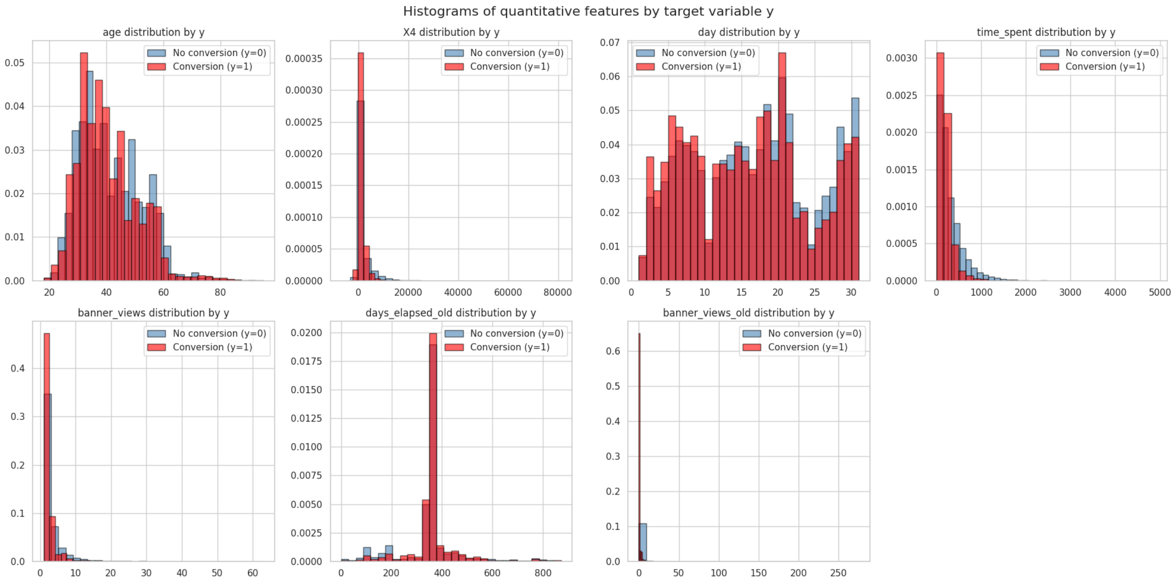
The targeted split has a mild imbalance, 57% non-conversion to 43% conversion. We shouldn't use specific resampling techniques here, but we do focus on a couple of key points:

1. Use stratified cross-validation to preserve class proportions in every fold.
2. Keep the baseline in mind: a naïve classifier predicting the majority class achieves only ~57% accuracy.

We proceed to visualise structure across quantitative and categorical features:

In [29]:
```
fig, axes = plt.subplots(2, 4, figsize=(20, 10))
```

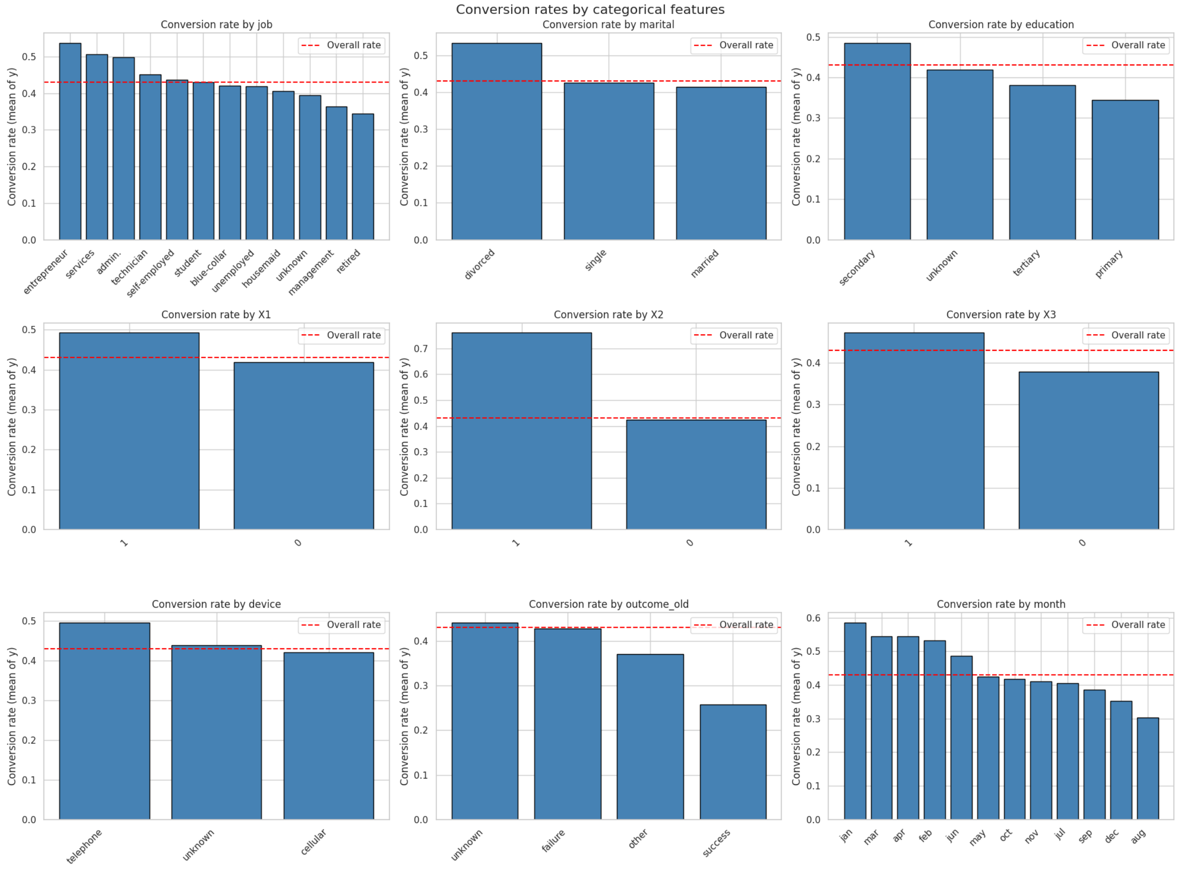
```
axes = axes.ravel()

for i, col in enumerate(quantitative):
    ax = axes[i]
    for yval, colour, label in [(0, 'steelblue', 'No conversion (y=0)'),
                                 (1, 'red', 'Conversion (y=1)')]:
        subset = train.loc[train['y'] == yval, col]
        ax.hist(subset, bins=30, alpha=0.6, color=colour, label=label, ed
    ax.set_title(f"{col} distribution by y")
    ax.legend()

axes[-1].set_visible(False)
plt.suptitle("Histograms of quantitative features by target variable y",
plt.tight_layout()
plt.show()
```



Quantitative structure takeaways

X4 is heavily right-skewed, so a log-transform may help with our linear-models. banner_views and banner_views_old are right-skewed. time_spent interestingly shows a significant shift between classes — non-conversion (y=0) spend more time. This seems to signal that quick engagement and attention-capture has a larger chance of conversion. age is roughly symmetric, with some class separation.

In [30]:
```
cat_vars = qualitative
fig, axes = plt.subplots(3, 3, figsize=(20, 15))
for ax, col in zip(axes.ravel(), cat_vars):
    rates = train.groupby(col)['y'].agg(['mean', 'count']).sort_values('m
    bars = ax.bar(range(len(rates)), rates['mean'], color='steelblue', ed
    ax.set_xticks(range(len(rates)))
    ax.set_xticklabels(rates.index, rotation=45, ha='right')
    ax.set_title(f"Conversion rate by {col}")
    ax.set_ylabel("Conversion rate (mean of y)")
    ax.axhline(train['y'].mean(), color='red', linestyle='--', label='Ove
    ax.legend()
plt.suptitle("Conversion rates by categorical features", fontsize=16)
plt.tight_layout()
plt.show()
```



Key categorical takeaways

outcome_old: "success" has the lowest conversion rate, around 26%. month: suggests seasonal variation, with Jan, Mar, April being the highest. Job: Students and Retirees convert at the largest rates.

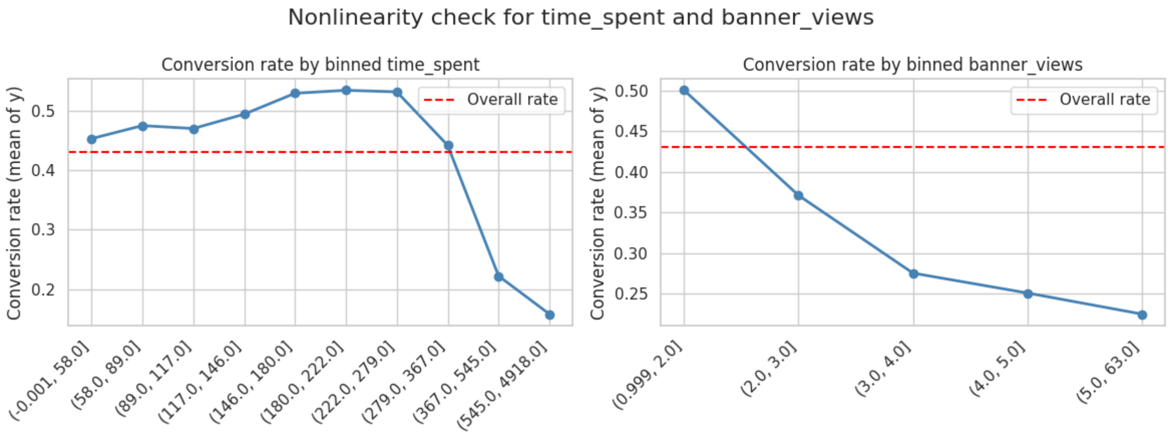
We proceed to check for correlation structure and nonlinearity:

In [31]:
```
# Pearson correlation of quantitative features with target variable y
corr = train[quantitative + ['y']].corr().drop('y').sort_values('y', asce
print("Pearson correlation of target variable y:")
for var, val in corr['y'].items():
    bar = '█' * int(abs(val) * 50)
    sign = '+' if val > 0 else '-'
    print(f"{var:20s} : {val:.4f} {sign} {bar}")
```

```
Pearson correlation of target variable y:
time_spent           : -0.2009 - ██████████
banner_views         : -0.1687 - ████████
X4                   : -0.0778 - ███
day                  : -0.0639 - ███
age                  : -0.0531 - ██
banner_views_old     : -0.0224 - █
days_elapsed_old     : 0.0542 + ██
```

In [32]:
```
# Nonlinearity check

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

for ax, col in zip(axes, ['time_spent', 'banner_views']):
    helper = train.copy()
    helper[col + '_bin'] = pd.qcut(helper[col], q=10, duplicates='drop')
```

```
    rates = helper.groupby(col + '_bin')['y'].mean()
    ax.plot(range(len(rates)), rates.values, 'o-', color='steelblue', lin
    ax.set_xticks(range(len(rates)))
    ax.set_xticklabels(rates.index.astype(str), rotation=45, ha='right')
    ax.set_title(f"Conversion rate by binned {col}")
    ax.set_ylabel("Conversion rate (mean of y)")
    ax.axhline(train['y'].mean(), color='red', linestyle='--', label='Ove
    ax.legend()
plt.suptitle("Nonlinearity check for time_spent and banner_views", fontsi
plt.tight_layout()
plt.show()
```



time_spent and banner_views show decreasing relationships that are monotonic with conversion. This pattern indicates that logistic regression and LDA may be suitable models. On the tails we have curvature, which indicates that kNN and QDA may capture signal at the extremes.

Dimensionality discussion

The curse of dimensionality is an important theoretical consideration. The number of predictors p is large relative to the structure of the data, expanding our dimensionality.

In [33]:

```
# Dimensionality count after one-hot encoding
var_counts = ['job', 'marital', 'education', 'device', 'outcome_old', 'mo
levels = sum(train[var].nunique() for var in var_counts)

dummies = sum(train[var].nunique() - 1 for var in var_counts)

n_numeric = len(quantitative)
n_binary = 3
n_engineered = 4

p_total = n_numeric + dummies + n_binary + n_engineered
n = len(train)

print(f"Total features after one-hot encoding: {p_total}")
print(f"Number of training samples: {n}")
print(f"n/p ratio: {n/p_total:.0f}")
```

```
Total features after one-hot encoding: 46
Number of training samples: 31647
n/p ratio: 688
```

The n/p ratio is comfortable for parametric models at 688, but the expanded dimensionality has consequences for kNN: as data points become approximately equidistant, the query point no longer remains local. This leads to kNN requiring a large k to stabilise predictions. We may find that Lasso logistic regression or Naive Bayes work better here.

Feature engineering and preprocessing

Quantitative variables are standardised using StandardScaler for kNN and regularised logistic regression. Qualitative variables are converted to dummy variables using one-hot encoding, with the first level dropped to avoid multicollinearity and "unknown" retained as a useful level.

In [34]:
```
def engineer_features(df):
    df = df.copy()
    df['old_campaign_exposed'] = (df['banner_views_old'] > 0).astype(int)
    df['old_success'] = (df['outcome_old'] == 'success').astype(int)
    df['time_per_view'] = df['time_spent'] / (df['banner_views'] + 1).cli
    df['X4_log'] = np.sign(df['X4']) * np.log1p(np.abs(df['X4']))  # Log-
    return df

train_featured = engineer_features(train)
test_featured = engineer_features(test)

print("New features added: old_campaign_exposed, old_success, time_per_vi
```

```
New features added: old_campaign_exposed, old_success, time_per_view, X4_l
og
```

In [35]:
```
numeric_features = quantitative + ['time_per_view', 'X4_log']
binary_features = ['old_campaign_exposed', 'old_success', 'X1', 'X2', 'X3
categorical_features = ['job', 'marital', 'education', 'device', 'outcome

all_features = numeric_features + binary_features + categorical_features
print(f"Total features after engineering: {len(all_features)}")

x_train = train_featured[all_features]
y_train = train_featured['y']
x_test = test_featured[all_features]
test_ids = test_featured['ID']

# Preprocessor pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('bin', 'passthrough', binary_features),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=Fals
    ]
)

print("Preprocessor pipeline created with standard scaling for numeric fe
print(f"feature breakdown: {len(numeric_features)} numeric, {len(binary_f
print(f"X_train shape before preprocessing: {x_train.shape}, X_test shape
```

```
Total features after engineering: 20
Preprocessor pipeline created with standard scaling for numeric features,
passthrough for binary features, and one-hot encoding for categorical feat
ures.
feature breakdown: 9 numeric, 5 binary, 6 categorical (one-hot encoded to
32 features)
X_train shape before preprocessing: (31647, 20), X_test shape before prepr
ocessing: (13564, 20)
```

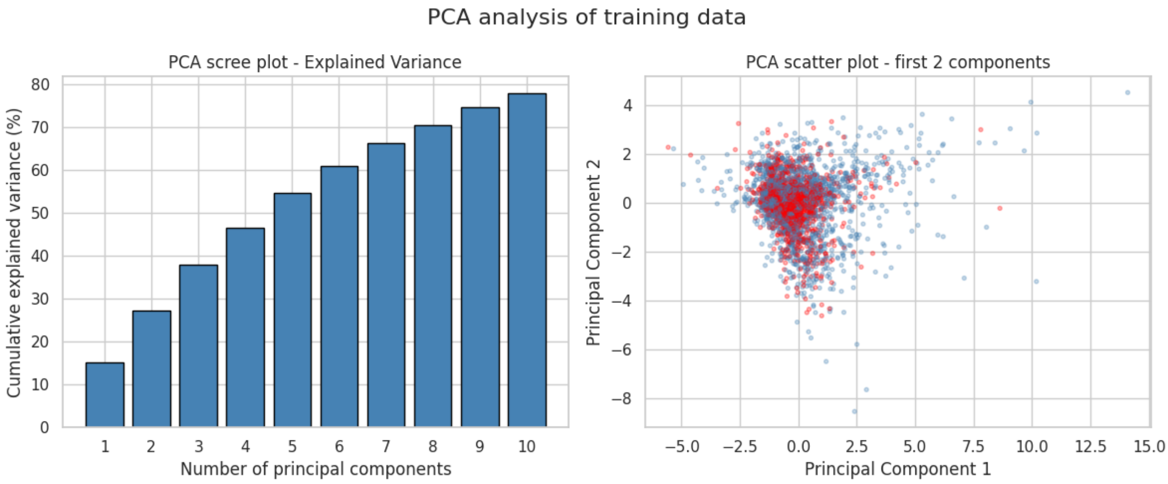
PCA Analysis

PCA was introduced in week 4 as an unsupervised tool to find maximal variance directions. We apply it to check if the data admits a low-dimensional structure.

In [36]:

```
X_train_processed = preprocessor.fit_transform(x_train)
print(f"X_train shape after preprocessing: {X_train_processed.shape}")

pca = PCA(n_components=10, random_state=SEED)
X_pca = pca.fit_transform(X_train_processed)
print(f"Shape of PCA-transformed training data: {X_pca.shape}")

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

ax = axes[0]
cum_var = np.cumsum(pca.explained_variance_ratio_) * 100
ax.bar = ax.bar(range(1, len(cum_var) + 1), cum_var, color='steelblue', e
ax.set_xticks(range(1, len(cum_var) + 1))
ax.set_xticklabels(range(1, len(cum_var) + 1))
ax.set_xlabel('Number of principal components')
ax.set_ylabel('Cumulative explained variance (%)')
ax.set_title('PCA scree plot - Explained Variance')

ax = axes[1]
colours = y_train.map({0: 'steelblue', 1: 'red'})
sample_idx = np.random.choice(len(X_pca), size=3000, replace=False)
ax.scatter(X_pca[sample_idx, 0], X_pca[sample_idx, 1], c=colours.iloc[sam
ax.set_xlabel('Principal Component 1')
ax.set_ylabel('Principal Component 2')
ax.set_title('PCA scatter plot - first 2 components')

plt.suptitle("PCA analysis of training data", fontsize=16)
plt.tight_layout()
plt.show()

print(f"Cumulative variance explained by 5 PCs: {cum_var[4]:.1f}%")
print(f"Cumulative variance explained by 10 PCs: {cum_var[9]:.1f}%")
```

```
X_train shape after preprocessing: (31647, 52)
Shape of PCA-transformed training data: (31647, 10)
```




```
Cumulative variance explained by 5 PCs: 54.5%
Cumulative variance explained by 10 PCs: 77.8%
```

The PCA scree plot confirms the high effective dimensionality — there is no clean separator in low dimensional projections.

EDA Recap

Finding	Implication for modelling
Moderate class imbalance (43% positive)	Use stratified CV; log-loss as evaluation metric
"unknown" encodes structural missingness	Retain as its own category, do not impute
No literal NaN in numeric columns	No numeric imputation needed
outcome_old = "success" has lowest conversion	Strong predictor; include as feature
time_spent, banner_views negatively correlated with y	Key numeric predictors; near-linear relationship
X4 heavily skewed	Apply log-transform
month, X1 show large group-level differences	Important categorical predictors
device mostly unknown	Low signal; include but expect low importance
High effective dimensionality (~46 after one-hot)	Curse of dimensionality affects kNN; favours Lasso for variable selection
PCA shows overlap, no dramatic nonlinearity	Linear models (LR, LDA) should be competitive

Step 2 - Model Building and Justification

We choose to train 6 models, covered by the course. All models are wrapped in a pipeline with consistent preprocessing:

1. **Logistic regression (Ridge/L2)** — directly estimates P(y=1|X) with L2

regularisation.

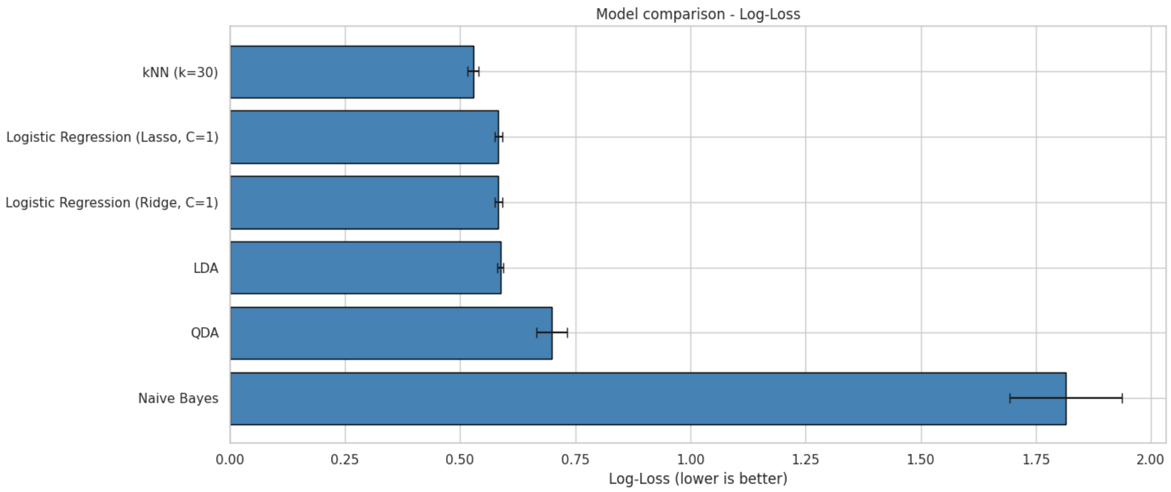
2. **Logistic regression (Lasso/L1)** — same as above but with an L1 penalty, performing automatic variable selection by shrinking irrelevant coefficients to 0. Useful due to unnamed and dummy variables.
3. **k-nearest neighbours** — non-parametric, requires standardised features.
4. **Linear Discriminant Analysis** — assumes each class has a multivariate Gaussian distribution with a shared covariance matrix.
5. **Quadratic Discriminant Analysis** — similar to LDA, but allows each class to have its own covariance matrix, leading to flexibility at the cost of more estimated parameters.
6. **Naive Bayes** — assumes predictors are independent given the class. This "naive" assumption reduces a p-dimensional estimation problem to p univariate problems, dramatically reducing variance at the cost of potentially increased bias if predictors are correlated.

In [37]:
```
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=SEED)

models = {
    'Logistic Regression (Ridge, C=1)': Pipeline([
        ('pre', preprocessor),
        ('clf', LogisticRegression(penalty='l2', C=1.0, max_iter=2000,
                                   solver='lbfgs', random_state=SEED))
    ]),
    'Logistic Regression (Lasso, C=1)': Pipeline([
        ('pre', preprocessor),
        ('clf', LogisticRegression(penalty='l1', C=1.0, max_iter=2000,
                                   solver='saga', random_state=SEED))
    ]),
    'kNN (k=30)': Pipeline([
        ('pre', preprocessor),
        ('clf', KNeighborsClassifier(n_neighbors=30))
    ]),
    'LDA': Pipeline([
        ('pre', preprocessor),
        ('clf', LinearDiscriminantAnalysis())
    ]),
    'QDA': Pipeline([
        ('pre', preprocessor),
        ('clf', QuadraticDiscriminantAnalysis(reg_param=0.5))
    ]),
    'Naive Bayes': Pipeline([
        ('pre', preprocessor),
        ('clf', GaussianNB())
    ]),
}

print(f"{'Model':<40s}  {'Log-Loss':>18s}  {'ROC-AUC':>18s}")
print('-' * 80)

results = {}
for name, pipe in models.items():
    ll_scores  = cross_val_score(pipe, x_train, y_train, cv=cv,
                                 scoring='neg_log_loss', n_jobs=-1)
    auc_scores = cross_val_score(pipe, x_train, y_train, cv=cv,
                                 scoring='roc_auc', n_jobs=-1)
```

```
    results[name] = {
        'log_loss_mean': -ll_scores.mean(),
        'log_loss_std':   ll_scores.std(),
        'auc_mean':       auc_scores.mean(),
        'auc_std':        auc_scores.std(),
    }
    print(f"{name:<40s}  {-ll_scores.mean():.4f} ± {ll_scores.std():.4f}
          f"{auc_scores.mean():.4f} ± {auc_scores.std():.4f}")
```

```
Model                                                 Log-Loss          R
OC-AUC
--------------------------------------------------------------------------
------
Logistic Regression (Ridge, C=1)          0.5834 ± 0.0085   0.7597 ± 0.010
0
Logistic Regression (Lasso, C=1)          0.5833 ± 0.0085   0.7598 ± 0.010
0
kNN (k=30)                                0.5281 ± 0.0123   0.8174 ± 0.010
1
LDA                                       0.5872 ± 0.0071   0.7524 ± 0.010
2
QDA                                       0.6983 ± 0.0334   0.7682 ± 0.010
8
Naive Bayes                               1.8148 ± 0.1212   0.7059 ± 0.011
5
```

In [38]:
```
res_df = pd.DataFrame(results).T.sort_values('log_loss_mean')

fig, ax = plt.subplots(figsize=(14, 6))
ax.barh(res_df.index, res_df['log_loss_mean'], xerr=res_df['log_loss_std'
        color='steelblue', edgecolor='black', capsize=4)
ax.set_xlabel('Log-Loss (lower is better)')
ax.set_title('Model comparison - Log-Loss')
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```



Initial comparison takeaways:

1. Ridge and Lasso Logistic regression perform similarly at C = 1. We will tune C for each separately.
2. LDA compares to logistic regression, as expected as we estimate similar linear boundaries.

3. Naive Bayes has a significant log-loss, indicating that bias may be introduced as predictors are correlated (time_spent, banner_views).
4. QDA seems to struggle with high-dimensional one-hot encoded features.
5. kNN has not been tuned yet — its performance depends on k.

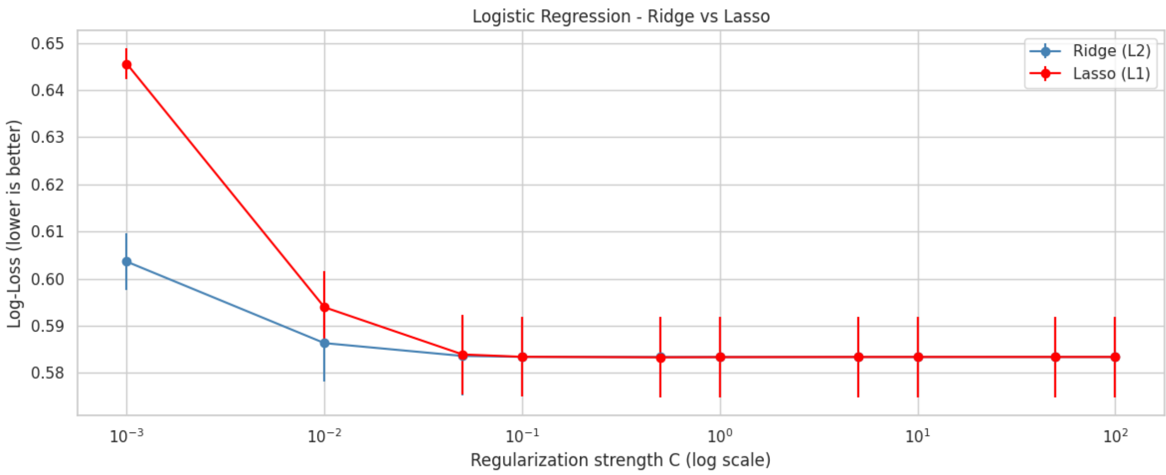
Step 3 - Validation and model selection

We use a 10-fold stratified cross-validation to tune hyperparameters, with the primary metric being log-loss to select models. We also use Bootstrap to quantify uncertainty for our performance metrics and assess whether differences are statistically meaningful.

Logistic Regression tuning (Ridge and Lasso):

In [39]:
```
C_values = [0.001, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0]
ridge_results, lasso_results = [], []

for C in C_values:
    pipe = Pipeline([('pre', preprocessor),
        ('clf', LogisticRegression(penalty='l2', C=C, max_iter=2000, solv
    scores = cross_val_score(pipe, x_train, y_train, cv=cv, scoring='neg_
    ridge_results.append({'C': C, 'log_loss_mean': -scores.mean(), 'log_l

for C in C_values:
    pipe = Pipeline([('pre', preprocessor),
        ('clf', LogisticRegression(penalty='l1', C=C, max_iter=2000, solv
    scores = cross_val_score(pipe, x_train, y_train, cv=cv, scoring='neg_
    lasso_results.append({'C': C, 'log_loss_mean': -scores.mean(), 'log_l

ridge_df = pd.DataFrame(ridge_results)
lasso_df = pd.DataFrame(lasso_results)
best_C_ridge = ridge_df.loc[ridge_df['log_loss_mean'].idxmin(), 'C']
best_C_lasso = lasso_df.loc[lasso_df['log_loss_mean'].idxmin(), 'C']
print(f"Best Ridge C = {best_C_ridge}, log-loss = {ridge_df['log_loss_mea
print(f"Best Lasso C = {best_C_lasso}, log-loss = {lasso_df['log_loss_mea

fig, ax = plt.subplots(figsize=(12, 5))
ax.errorbar(ridge_df['C'], ridge_df['log_loss_mean'], yerr=ridge_df['log_
ax.errorbar(lasso_df['C'], lasso_df['log_loss_mean'], yerr=lasso_df['log_
ax.set_xscale('log')
ax.set_xlabel('Regularization strength C (log scale)')
ax.set_ylabel('Log-Loss (lower is better)')
ax.set_title('Logistic Regression - Ridge vs Lasso')
ax.legend()
plt.tight_layout()
plt.show()
```

```
Best Ridge C = 0.5, log-loss = 0.58335
Best Lasso C = 0.5, log-loss = 0.58331
```



Model does not seem overtly sensitive to regularisation. Since the best log-loss results are similar, Lasso is preferred for its variable selection properties, leading to a more interpretable model. We inspect which coefficients Lasso shrunk to zero:

In [40]:
```
# Fit lasso on best C to inspect coefficients
lasso_inspect = Pipeline([
    ('pre', preprocessor),
    ('clf', LogisticRegression(penalty='l1', C=best_C_lasso, max_iter=200
                               solver='saga', random_state=SEED))
])
lasso_inspect.fit(x_train, y_train)

ohe = lasso_inspect.named_steps['pre'].named_transformers_['cat']
cat_feature_names = ohe.get_feature_names_out(categorical_features).tolis
feature_names = numeric_features + binary_features + cat_feature_names

lasso_coefs = pd.Series(lasso_inspect.named_steps['clf'].coef_[0], index=
n_zero = (lasso_coefs == 0).sum()
n_nonzero = (lasso_coefs != 0).sum()

print(f"Lasso zeroed out {n_zero} of {len(lasso_coefs)} coefficients ({10
print(f"\nTop 10 non-zero coefficients by magnitude:")
surviving = lasso_coefs[lasso_coefs != 0].reindex(
    lasso_coefs[lasso_coefs != 0].abs().sort_values(ascending=False).inde
for feat, coef in surviving.head(10).items():
    print(f"  {feat:30s} : {coef:+.4f}")
```

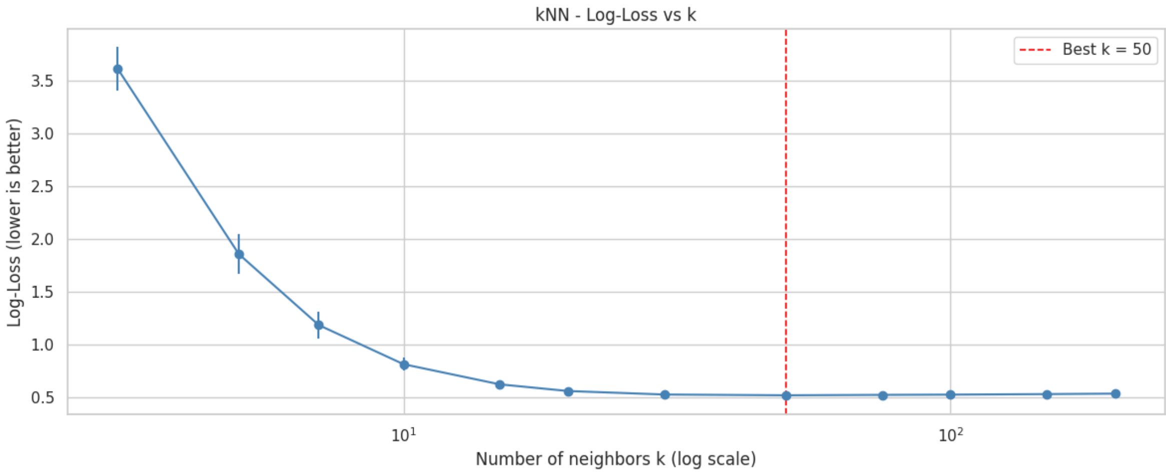
```
Lasso zeroed out 3 of 52 coefficients (6%)

Top 10 non-zero coefficients by magnitude:
  X2                             : +2.2372
  time_spent                     : -0.9016
  banner_views                   : -0.5902
  month_mar                      : +0.5135
  month_jan                      : +0.5109
  month_aug                      : -0.5068
  X3                             : +0.4761
  month_nov                      : -0.4630
  X4_log                         : +0.4339
  device_telephone               : +0.4264
```

Tuning kNN

The choice of k directly controls the bias-variance trade-off. Given the curse of dimensionality discussion, we expect kNN to need a relatively large k in this ~50-dimensional feature space.

In [41]:
```
k_values = [3, 5, 7, 10, 15, 20, 30, 50, 75, 100, 150, 200]
knn_results = []
for k in k_values:
    pipe = Pipeline([('pre', preprocessor), ('clf', KNeighborsClassifier(
    scores = cross_val_score(pipe, x_train, y_train, cv=cv, scoring='neg_
    knn_results.append({'k': k, 'log_loss_mean': -scores.mean(), 'log_los

knn_df = pd.DataFrame(knn_results)
best_k = knn_df.loc[knn_df['log_loss_mean'].idxmin(), 'k']
print(f"Best kNN - k={best_k}, log-loss = {knn_df['log_loss_mean'].min():

fig, ax = plt.subplots(figsize=(12, 5))
ax.errorbar(knn_df['k'], knn_df['log_loss_mean'], yerr=knn_df['log_loss_s
ax.set_xscale('log')
ax.axvline(best_k, ls='--', color='red', lw=1.2, label=f'Best k = {best_k
ax.set_xlabel('Number of neighbors k (log scale)')
ax.set_ylabel('Log-Loss (lower is better)')
ax.set_title('kNN - Log-Loss vs k')
ax.legend()
plt.tight_layout()
plt.show()
```

```
Best kNN - k=50, log-loss = 0.5208
```



As anticipated, the optimal k is relatively large. Small k overfits due to high variance from noisy estimates, while large k underfits. This confirms that the high-dimensional space isn't truly local, and the model averages over many points to get stable estimates.

LDA, QDA and Naive Bayes tuning

In [42]:
```
# LDA (no hyperparameters to tune)
lda_pipe = Pipeline([('pre', preprocessor), ('clf', LinearDiscriminantAna
lda_scores = cross_val_score(lda_pipe, x_train, y_train, cv=cv, scoring='
lda_log_loss_mean = -lda_scores.mean()
print(f"LDA - log-loss: {lda_log_loss_mean:.4f} ± {lda_scores.std():.4f}"

# QDA with reg_param tuning (start from 0.1 to avoid singular covariance)
reg_params = [0.1, 0.25, 0.5, 0.75, 0.9]
qda_results = []
```

```
for reg in reg_params:
    pipe = Pipeline([('pre', preprocessor), ('clf', QuadraticDiscriminant
    scores = cross_val_score(pipe, x_train, y_train, cv=cv, scoring='neg_
    qda_results.append({'reg_param': reg, 'log_loss_mean': -scores.mean()

qda_df = pd.DataFrame(qda_results)
best_reg = qda_df.loc[qda_df['log_loss_mean'].idxmin(), 'reg_param']
best_ll_qda = qda_df['log_loss_mean'].min()
print(f"Best QDA - reg_param={best_reg}, log-loss = {best_ll_qda:.4f}")
if lda_log_loss_mean < best_ll_qda:
    print("LDA outperforms QDA. The extra flexibility of QDA is not justi

# Naive Bayes with variance smoothing search
var_smoothing_values = [1e-12, 1e-9, 1e-6, 1e-3, 1e-1, 1.0]
nb_results = []
for vs in var_smoothing_values:
    pipe = Pipeline([('pre', preprocessor), ('clf', GaussianNB(var_smooth
    scores = cross_val_score(pipe, x_train, y_train, cv=cv, scoring='neg_
    nb_results.append({'var_smoothing': vs, 'log_loss_mean': -scores.mean

nb_df = pd.DataFrame(nb_results)
best_var_smooth = nb_df.loc[nb_df['log_loss_mean'].idxmin(), 'var_smoothi
print(f"Best Naive Bayes - var_smoothing={best_var_smooth:.0e}, log-loss
```

```
LDA - log-loss: 0.5872 ± 0.0071
Best QDA - reg_param=0.9, log-loss = 0.6257
LDA outperforms QDA. The extra flexibility of QDA is not justified here.
Best Naive Bayes - var_smoothing=1e+00, log-loss = 0.7534
```

Comparison of all tuned models

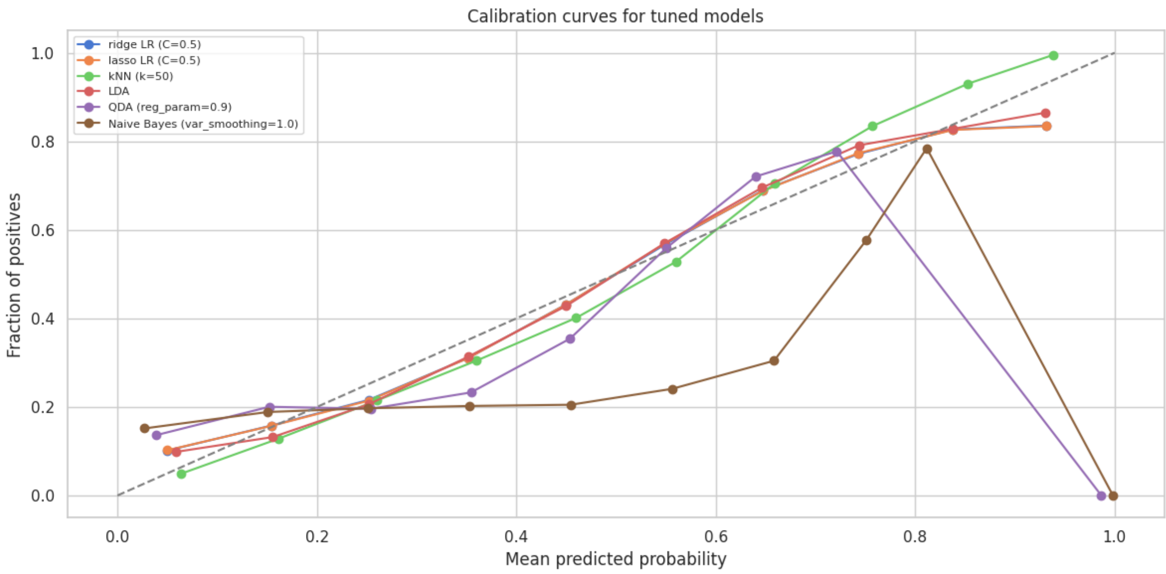
In [43]:
```
tuned_models = {
    f"ridge LR (C={best_C_ridge})": Pipeline([
        ('pre', preprocessor),
        ('clf', LogisticRegression(penalty='l2', C=best_C_ridge, max_iter
                                   solver='lbfgs', random_state=SEED))
    ]),
    f"lasso LR (C={best_C_lasso})": Pipeline([
        ('pre', preprocessor),
        ('clf', LogisticRegression(penalty='l1', C=best_C_lasso, max_iter
                                   solver='saga', random_state=SEED))
    ]),
    f"kNN (k={best_k})": Pipeline([
        ('pre', preprocessor),
        ('clf', KNeighborsClassifier(n_neighbors=best_k))
    ]),
    "LDA": Pipeline([
        ('pre', preprocessor),
        ('clf', LinearDiscriminantAnalysis())
    ]),
    f"QDA (reg_param={best_reg})": Pipeline([
        ('pre', preprocessor),
        ('clf', QuadraticDiscriminantAnalysis(reg_param=best_reg))
    ]),
    f"Naive Bayes (var_smoothing={best_var_smooth})": Pipeline([
        ('pre', preprocessor),
        ('clf', GaussianNB(var_smoothing=best_var_smooth))
    ]),
}
```

```
print(f"{'Model':<40s}  {'Log-Loss':>18s}")
print('-' * 60)

final_results = {}
for name, pipe in tuned_models.items():
    ll_scores  = cross_val_score(pipe, x_train, y_train, cv=cv,
                                 scoring='neg_log_loss', n_jobs=-1)
    final_results[name] = {
        'log_loss_mean': -ll_scores.mean(),
        'log_loss_std':   ll_scores.std(),
    }
    print(f"{name:<40s}  {-ll_scores.mean():.4f} ± {ll_scores.std():.4f}"

    best_model = min(final_results, key=lambda x: final_results[x]['log_l
print(f"\nBest overall model: {best_model} with log-loss = {final_results
```

```
Model                                                   Log-Loss
------------------------------------------------------------
ridge LR (C=0.5)                             0.5834 ± 0.0085
lasso LR (C=0.5)                             0.5833 ± 0.0085
kNN (k=50)                                   0.5208 ± 0.0099
LDA                                          0.5872 ± 0.0071
QDA (reg_param=0.9)                          0.6257 ± 0.0119
Naive Bayes (var_smoothing=1.0)              0.7534 ± 0.0261

Best overall model: kNN (k=50) with log-loss = 0.5208 ± 0.0099
```

In [44]:

```
# Calibration curves
fig, ax = plt.subplots(figsize=(12, 6))
for name, pipe in tuned_models.items():
    y_prob = cross_val_predict(pipe, x_train, y_train, cv=cv, method='pre
    prob_true, prob_pred = calibration_curve(y_train, y_prob, n_bins=10)
    ax.plot(prob_pred, prob_true, marker='o', label=name)
ax.plot([0, 1], [0, 1], ls='--', color='gray')
ax.set_xlabel('Mean predicted probability')
ax.set_ylabel('Fraction of positives')
ax.set_title('Calibration curves for tuned models')
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```



Calibration analysis takeaways

Logistic Regression (both Ridge and Lasso) produces well-calibrated probabilities by construction. kNN probabilities are inherently coarse (multiples of 1/k), which can hurt log-loss. Naive Bayes may produce overconfident probabilities due to the independence assumption compounding evidence more aggressively than warranted when predictors are correlated.

Model selction takeaways

kNN K=50 emerges as the best model by CV log-loss, suprising considering our earlier discussions on the curse of dimensionality. Despite the aproxximately 50 dimensional feature space, kNN outperforms the parametric models. This suggests that the data contains local nonlinera structure that LR and LDA cannot capture, even though the binned conversion rate plots in the EDA section were relatively monotonic. The large optimal k at k=50 still confirms that kNN is compensating for the high dimensionality by averaging over many neighbours, but this average still preserves enough local signal to beat the rest of the models.

Bootstrapping

We use bootstrapping to quantify the uncertainty in our estimates. We draw B=200 samples with replacement from the training data, fit the models, and evaluate on out-of-bag observations.

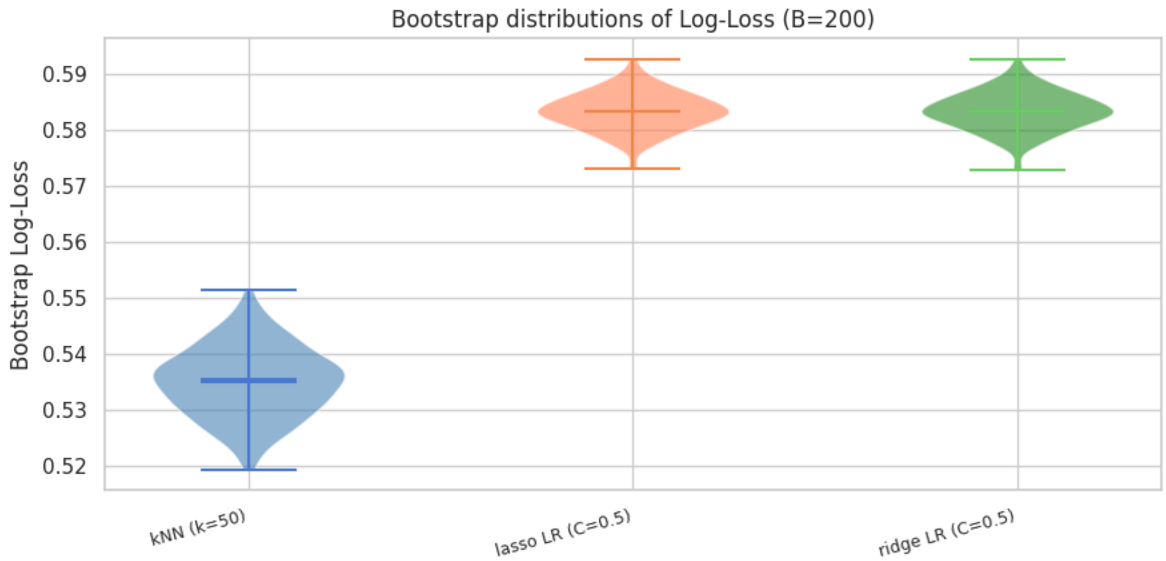
In [45]:
```
def bootstrap_log_loss(pipe, X, y, B=200, seed=42):
    rng = np.random.RandomState(seed)
    n = len(X)
    X = X.reset_index(drop=True)
    y = y.reset_index(drop=True)
    boot_losses = []

    for b in range(B):
        idx_boot = rng.choice(n, size=n, replace=True)
        idx_oob = np.setdiff1d(range(n), idx_boot)
        if len(idx_oob) < 10:
            continue
        X_boot, y_boot = X.iloc[idx_boot], y.iloc[idx_boot]
        X_oob, y_oob = X.iloc[idx_oob], y.iloc[idx_oob]
        if len(y_oob.unique()) < 2 or len(y_boot.unique()) < 2:
            continue
        try:
            pipe_clone = clone(pipe)
            pipe_clone.fit(X_boot, y_boot)
            y_oob_prob = pipe_clone.predict_proba(X_oob)[:, 1]
            boot_losses.append(log_loss(y_oob, y_oob_prob))
        except Exception:
            continue
    return np.array(boot_losses)

top_models = sorted(final_results.items(), key=lambda x: x[1]['log_loss_m
bootstrap_results = {}
for name, _ in top_models:
    print(f"Running bootstrap for {name}...")
    boot_losses = bootstrap_log_loss(tuned_models[name], x_train, y_train
    bootstrap_results[name] = boot_losses
```

```
    print(f"  log-loss: {boot_losses.mean():.5f} ± {boot_losses.std():.5f
```

```
Running bootstrap for kNN (k=50)...
  log-loss: 0.53505 ± 0.00624
Running bootstrap for lasso LR (C=0.5)...
  log-loss: 0.58343 ± 0.00344
Running bootstrap for ridge LR (C=0.5)...
  log-loss: 0.58347 ± 0.00345
```

In [46]:

```
# Visualise bootstrap distributions
colours = ['steelblue', 'coral', 'forestgreen']
fig, ax = plt.subplots(figsize=(9, 4.5))

for i, (name, boot_ll) in enumerate(bootstrap_results.items()):
    parts = ax.violinplot(boot_ll, positions=[i], showmeans=True, showmed
    for pc in parts['bodies']:
        pc.set_facecolor(colours[i])
        pc.set_alpha(0.6)

ax.set_xticks(range(len(bootstrap_results)))
ax.set_xticklabels(bootstrap_results.keys(), fontsize=9, rotation=15, ha=
ax.set_ylabel('Bootstrap Log-Loss')
ax.set_title('Bootstrap distributions of Log-Loss (B=200)')
plt.tight_layout()
plt.show()

# Report on overlap
names = list(bootstrap_results.keys())
if len(names) >= 2:
    ll_1 = bootstrap_results[names[0]]
    ll_2 = bootstrap_results[names[1]]
    diff = ll_2 - ll_1[:len(ll_2)]
    pct_better = (diff > 0).mean() * 100
    print(f"\nIn {pct_better:.0f}% of bootstrap samples, {names[0]} has l
    print(f"Mean difference: {diff.mean():.5f} ± {diff.std():.5f}")
    if diff.mean() - 1.96 * diff.std() > 0:
        print("The difference is statistically significant at the 95% con
    else:
        print("The difference is not statistically significant at the 95%
```

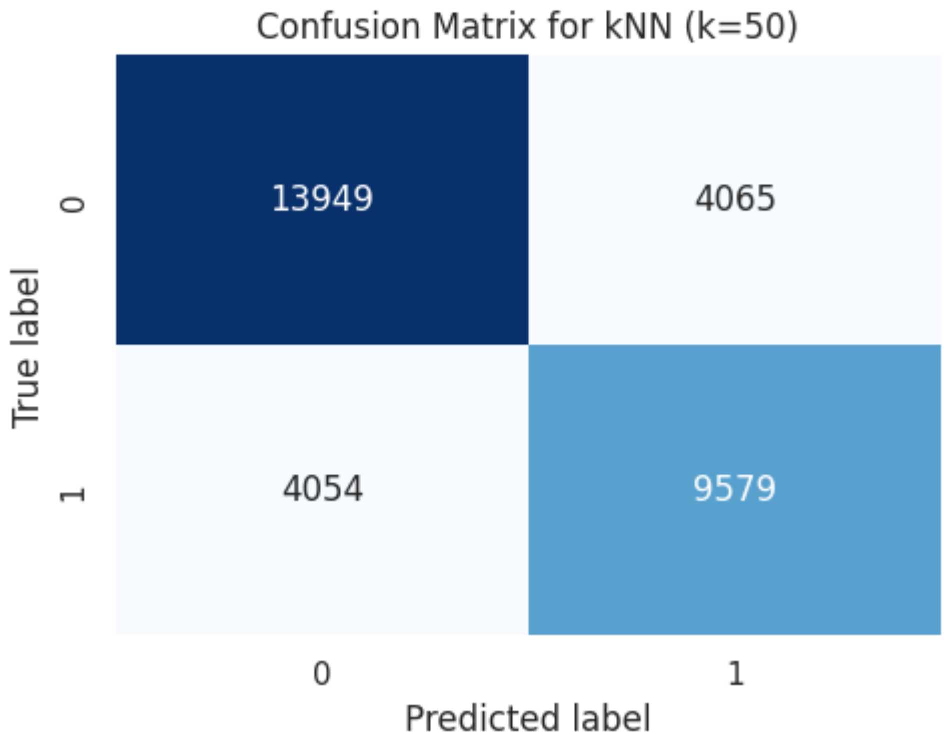


```
In 100% of bootstrap samples, kNN (k=50) has lower log-loss than lasso LR
(C=0.5).
Mean difference: 0.04838 ± 0.00598
The difference is statistically significant at the 95% confidence level.
```

The bootstrap analysis provides a full sampling distribution of the log-loss for each model. If the distributions of two models overlap substantially, the performance difference may not be statistically meaningful.

Confusion matrix of the selected model

In [47]:
```
best_pipe = tuned_models[best_model]
prob_cv = cross_val_predict(best_pipe, x_train, y_train, cv=cv, method='p
pred_cv = (prob_cv >= 0.5).astype(int)

cm = confusion_matrix(y_train, pred_cv)
fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=ax)
ax.set_xlabel('Predicted label')
ax.set_ylabel('True label')
ax.set_title(f'Confusion Matrix for {best_model}')
plt.tight_layout()
plt.show()
```



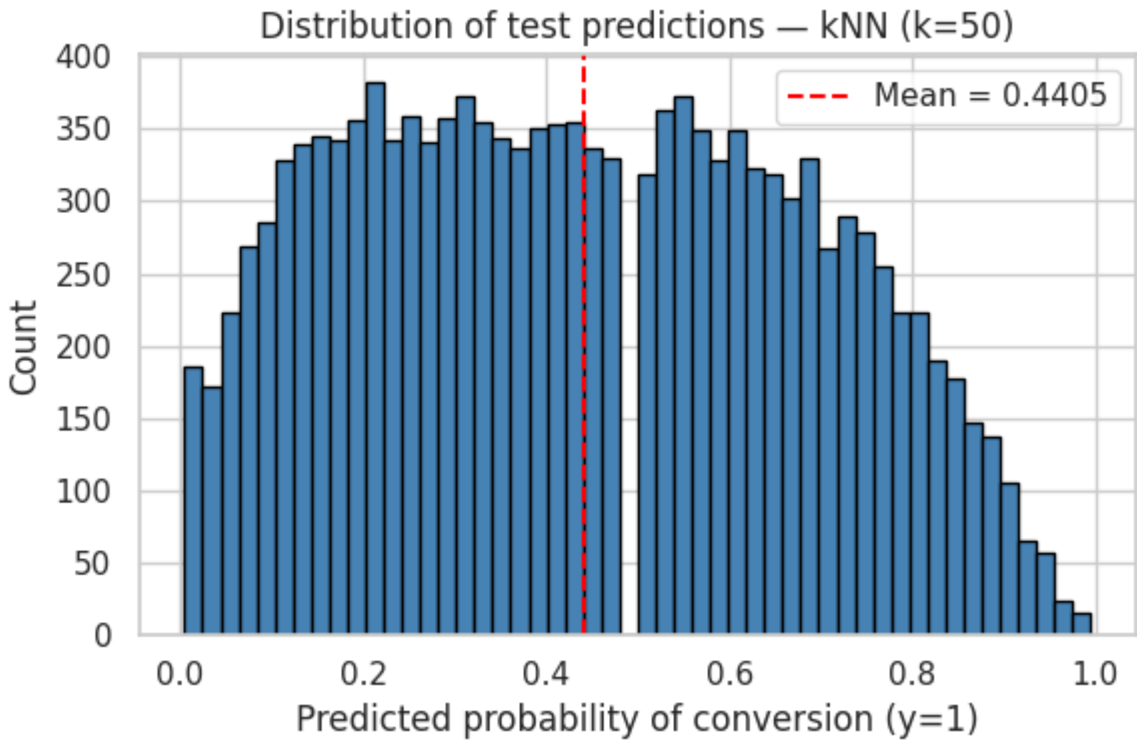
The confusion matrix is nearly symmetric — the model doesn't systematically favour predicting one class over another, which is reassuring given the mild class imbalance.

Step 4: Generating test predictions

We refit the selected model on the full training set and produce predicted probabilities. Predictions are clipped to [0.005, 0.995] to guard against the unbounded penalty that log-loss assigns to extreme probabilities.

In [48]:
```
print(f"Refitting best model {best_model} on full training data...")
best_pipe.fit(x_train, y_train)

p_hat = best_pipe.predict_proba(x_test)[:, 1]
p_hat = np.clip(p_hat, 0.005, 0.995)

print(f"\nNumber of predictions: {len(p_hat)}")
print(f"Min probability: {p_hat.min():.5f}")
print(f"Max probability: {p_hat.max():.5f}")
print(f"Mean predicted probability: {p_hat.mean():.5f}")

fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(p_hat, bins=50, color='steelblue', edgecolor='black')
ax.axvline(p_hat.mean(), color='red', linestyle='--', label=f'Mean = {p_h
ax.set_xlabel('Predicted probability of conversion (y=1)')
ax.set_title(f'Distribution of test predictions — {best_model}')
ax.set_ylabel('Count')
ax.legend()
plt.tight_layout()
plt.show()
```

```
Refitting best model kNN (k=50) on full training data...

Number of predictions: 13564
Min probability: 0.00500
Max probability: 0.99500
Mean predicted probability: 0.44054
```



In [49]:
```
submission = pd.DataFrame({'ID': test_ids, 'p_hat': p_hat})
```

```
submission.to_csv('predictions.csv', index=False)
print(f"Submission saved to 'predictions.csv' — shape: {submission.shape}
submission.head(10)
```

```
Submission saved to 'predictions.csv' — shape: (13564, 2)
```

Out[49]:

	ID	p_hat
0	1	0.66
1	2	0.80
2	3	0.84
3	4	0.50
4	5	0.14
5	6	0.86
6	7	0.36
7	8	0.68
8	9	0.16
9	10	0.30

Conclusion and Reflection

What worked well

Logistic Regression (both Ridge and Lasso) proved to be strong baselines, producing well-calibrated probabilities and competitive log-loss. The fact that it matches or outperforms more complex models reflects the roughly linear relationships identified in the EDA. Lasso's variable selection identified which of the many one-hot encoded features are predictive, yielding a sparser and more interpretable model while maintaining equivalent predictive performance to Ridge.

Feature engineering (e.g., `time_per_view`, `old_success` flag, log-transform of `X4`) provided consistent improvements. 10-fold stratified CV combined with Bootstrap uncertainty estimation allowed us to infer performance estimates for model selection accurately.

What did not work as well

kNN suffered from the curse of dimensionality: in the ~50-dimensional feature space, neighbourhoods became non-local, requiring a large k that smoothed out the flexibility that makes kNN attractive. Its coarse probability estimates also hurt log-loss. That being said, it still provided us with the best model, capturing enough non-linear signals that the parametric models could not, despite its faults. However, when inspecting the final data, we found that the course-estimates led to extreme log-loss penalities on the test sets (near 0 predictors), leading to values needing to be clipped.

QDA was also impacted by the high dimensionality — estimating separate covariance matrices per class required heavy regularisation that effectively pushed it back towards LDA-like behaviour. This meant that LDA was favourable, providing near identical log-loss without needing heavy computation for calibration.

Naive Bayes achieved reasonable discrimination but its independence assumption — violated by correlated engagement features like time_spent and banner_views — led to poorly calibrated probabilities, inflating log-loss.

Possible improvements

As logistic regression provided a suitable baseline and still has potential to capture extra signals, this would feel like the next thing to focus on. Interaction terms or polynomial features could capture nonlinearities within regression and its framework, perhaps beating kNN as a predictor.

If we wanted to stick to our best model, target encoding for high-cardinality categoricals like `job` and `month` could reduce dimensionality, potentially improving kNN and also QDA. Principal Components Regression (PCR) or Partial Least Squares (PLS) from Week 6 could address the high dimensionality more directly. Finally, ensemble methods (e.g., soft-voting over Logistic Regression, LDA, and Naive Bayes) could combine diverse model perspectives, giving us an even more accurate result.